REPRESENTING DATA

DATASET CLEANING AND CLASSIFICATION

STREAM 1

The group members:

Nour el houda Dabbabi
Eya Abdelhamid
Nour el houda ben Mohamed
Ghada Mhirsi
Ileys Megdiche

WHAT ARE THE PROJECT GOALS?

The goal is very simple:

All we need to do is to clean and classify a dataset of countries by continents and analyze their economic growth.

By following ceratin steps like:
splitting data by continent, cleaning and enhancing the dataset, creating a variable, using different loops(for,if,while...) ,
my group and i were able to achieve the “first stream” of cleaning and organizing the dataset.

WHY IS IT USEFUL?

ORGANISE INFORMATION

COMPARE DATA

MAKE ACCURATE DECISIONS

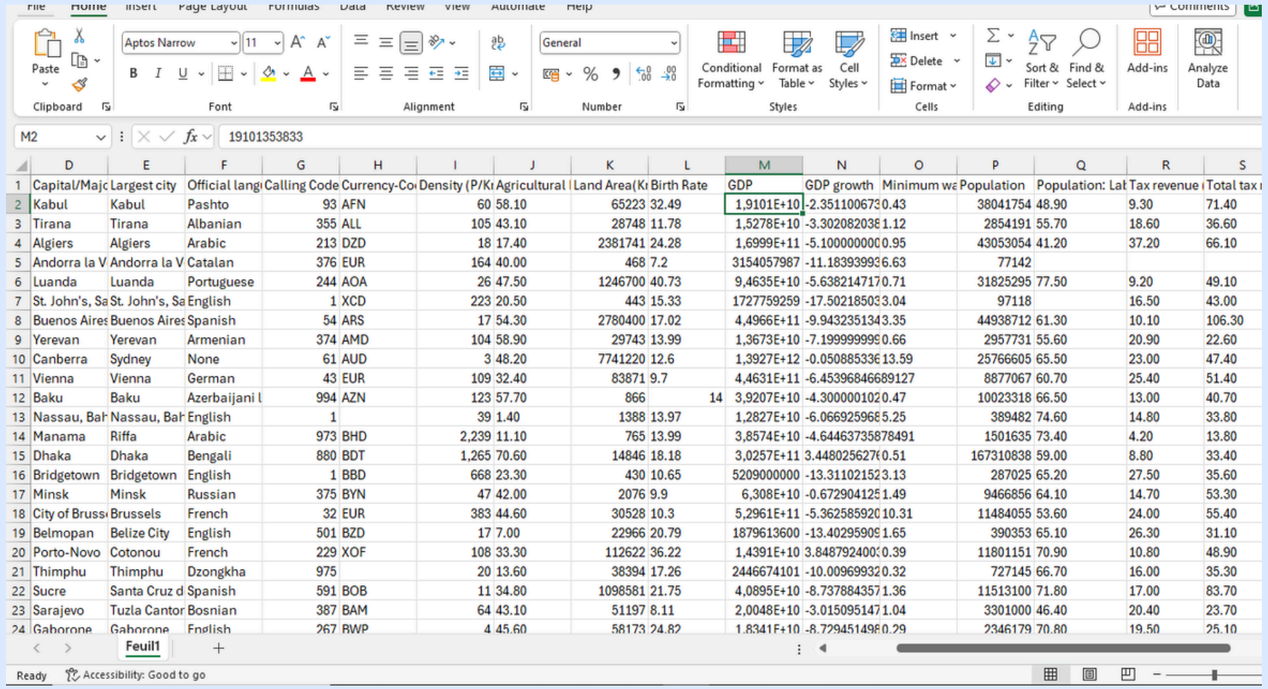
Let’s explore the different steps in detail !

Stream 1

OVERVIEW

Here’s a small representation of our initial dataset

Seeing this doesn’t really allow you to understand much that’s why we needed to clean it and organise it.



	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1	Capital/Majc	Largest city	Official langu	Calling Code	Currency-Co	Density (P/Kı	Agricultural l	Land Area(Kı	Birth Rate	GDP	GDP growth	Minimum wa	Population	Population: Lat	Tax revenue	Total tax
2	Kabul	Kabul	Pashto	93	AFN	60	58.10	65223	32.49	1,9101E+10	-2.351100673	0.43	38041754	48.90	9.30	71.40
3	Tirana	Tirana	Albanian	355	ALL	105	43.10	28748	11.78	1,5278E+10	-3.302082038	1.12	2854191	55.70	18.60	36.60
4	Algiers	Algiers	Arabic	213	DZD	18	17.40	2381741	24.28	1,6999E+11	-5.100000000	0.95	43053054	41.20	37.20	66.10
5	Andorra la V	Andorra la V	Catalan	376	EUR	164	40.00	468	7.2	3154057987	-11.18393993	6.63	77142			
6	Luanda	Luanda	Portuguese	244	AOA	26	47.50	1246700	40.73	9,4635E+10	-5.638214717	0.71	31825295	77.50	9.20	49.10
7	St. John's, Sa	St. John's, Sa	English	1	XCD	223	20.50	443	15.33	1727759259	-17.50218503	3.04	97118		16.50	43.00
8	Buenos Aires	Buenos Aires	Spanish	54	ARS	17	54.30	2780400	17.02	4,4966E+11	-9.943235134	3.35	44938712	61.30	10.10	106.30
9	Yerevan	Yerevan	Armenian	374	AMD	104	58.90	29743	13.99	1,3673E+10	-7.199999999	0.66	2957731	55.60	20.90	22.60
10	Canberra	Sydney	None	61	AUD	3	48.20	7741220	12.6	1,3927E+12	-0.050885336	13.59	25766605	65.50	23.00	47.40
11	Vienna	Vienna	German	43	EUR	109	32.40	83871	9.7	4,4631E+11	-6.453968466	89127	8877067	60.70	25.40	51.40
12	Baku	Baku	Azerbaijani l	994	AZN	123	57.70	866	14	3,9207E+10	-4.300000102	0.47	10023318	66.50	13.00	40.70
13	Nassau, Bah	Nassau, Bah	English	1		39	1.40	1388	13.97	1,2827E+10	-6.066925968	5.25	389482	74.60	14.80	33.80
14	Manama	Riffa	Arabic	973	BHD	2,239	11.10	765	13.99	3,8574E+10	-4.644637358	78491	1501635	73.40	4.20	13.80
15	Dhaka	Dhaka	Bengali	880	BDT	1,265	70.60	14846	18.18	3,0257E+11	3.448025627	0.51	167310838	59.00	8.80	33.40
16	Bridgetown	Bridgetown	English	1	BBD	668	23.30	430	10.65	5209000000	-13.31102152	3.13	287025	65.20	27.50	35.60
17	Minsk	Minsk	Russian	375	BYN	47	42.00	2076	9.9	6,308E+10	-0.672904125	1.49	9466856	64.10	14.70	53.30
18	City of Brusse	Brussels	French	32	EUR	383	44.60	30528	10.3	5,2961E+11	-5.362585920	10.31	11484055	53.60	24.00	55.40
19	Belmopan	Belize City	English	501	BZD	17	7.00	22966	20.79	1879613600	-13.40295909	1.65	390353	65.10	26.30	31.10
20	Porto-Novo	Cotonou	French	229	XOF	108	33.30	112622	36.22	1,4391E+10	3.848792400	0.39	11801151	70.90	10.80	48.90
21	Thimphu	Thimphu	Dzongkha	975		20	13.60	38394	17.26	2446674101	-10.00969932	0.32	727145	66.70	16.00	35.30
22	Sucre	Santa Cruz d	Spanish	591	BOB	11	34.80	1098581	21.75	4,0895E+10	-8.737884357	1.36	11513100	71.80	17.00	83.70
23	Sarajevo	Tuzla Cantor	Bosnian	387	BAM	64	43.10	51197	8.11	2,0048E+10	-3.015095147	1.04	3301000	46.40	20.40	23.70
24	Gaborone	Gaborone	English	267	BWP	4	45.60	58173	24.82	1,8341E+10	-8.729451498	0.29	2346179	70.80	19.50	25.10

STEP 1:

THE CODE:

```python
import pandas as pd

# Load the dataset
file_path = "basededonnee2.xlsx"
df = pd.read_excel(file_path)

# Initialize empty DataFrames for continents
df_africa = pd.DataFrame()
df_europe = pd.DataFrame()
df_asia = pd.DataFrame()
df_america_oceania = pd.DataFrame()
```

```python
# Function to classify countries by continent
def classify_by_continent(df):
    global df_africa, df_europe, df_asia, df_america_oceania  # Ensure global update of DataFrames
    for _, row in df.iterrows():
        continent = row['Continent']  # Access the 'Continent' column correctly
        if continent == 'Africa':
            df_africa = pd.concat([df_africa, pd.DataFrame([row.to_dict()])], ignore_index=True)
        elif continent == 'Europe':
            df_europe = pd.concat([df_europe, pd.DataFrame([row.to_dict()])], ignore_index=True)
        elif continent == 'Asia':
            df_asia = pd.concat([df_asia, pd.DataFrame([row.to_dict()])], ignore_index=True)
        elif continent in ['America', 'Oceania']:
            df_america_oceania = pd.concat([df_america_oceania, pd.DataFrame([row.to_dict()])], ignore_index=True)
```

Importating the dataset and dataframes creation

1- we imported the intial dataset using the Pandas library
2-we started by creating empty dataframes where we can save the data for each one of them.
3-we classifed countries and their indices by continent.

STEP 2:

Populating dataframes and deleting unwanted data

1- We grouped data by continent and we divided the dataset into four separate dataframes, one for each continent.
2- We excluded any unnecessary data like: the GDP and Continent columns to simplify the data.
3-We name each dataframe after its respective continent for clarity and organization.

```
# Call the function to classify by continent
classify_by_continent(df)
```

```
# Drop GDP and Continent columns from each continent-specific DataFrame
df_africa = df_africa.drop(columns=['Continent', 'GDP'], errors='ignore')
df_europe = df_europe.drop(columns=['Continent', 'GDP'], errors='ignore')
df_asia = df_asia.drop(columns=['Continent', 'GDP'], errors='ignore')
df_america_oceania = df_america_oceania.drop(columns=['Continent', 'GDP'], errors='ignore')
```

STEP 3: Data cleaning and filling missing information

1- Our proffessor assigned us the continent “America-oceania”
2-we started by creating empty dataframes where we can save the data for each of the coutries.
3-we classifed countries and their indices by the assigned continent.

```
# Replace missing values in numeric columns with their respective means
missing_values_america= df_america_oceania.isnull().sum()
print("\nMissing values in American countries data:")
print(missing_values_america[missing_values_america > 0])

numeric_cols = df_america_oceania.select_dtypes(include=['number']).columns
df_america_oceania[numeric_cols] = df_america_oceania[numeric_cols].fillna(df_america_oceania[numeric_cols].mean())
```

LAST STEP: New variable creation and GDP categorazing

1- We created a new variable to categorize economic growth levels based on GDP growth

2-we organized the GDP based off of rates ranging from 1% to 5%

```python
# GDP Growth Classification
def classify_growth(gdp):
    if gdp <= 1:
        return 'Very Low'
    elif 1 < gdp <= 3:
        return 'Low'
    elif 3 < gdp <= 5:
        return 'Medium'
    else:
        return 'High'
```

```python
# Add classification column
df_america_oceania['Economic_growth_level'] = df_america_oceania['GDP growth'].apply(classify_growth)

print("\nAmerican countries with classified GDP growth:")
print(df_america_oceania[['Country', 'GDP growth', 'Economic_growth_level']])
```

```python
# Ensure GDP Growth column is numeric and handle missing values
df_america_oceania['GDP growth'] = pd.to_numeric(df_asia['GDP growth'], errors='coerce')
df_america_oceania['GDP growth'] = df_america_oceania['GDP growth'].fillna(df_america_oceania['GDP growth'].mean())
```

RESULT:

We printed the country, gdp growth,economic growth level to show the result:

```
print("\nAmerican countries with classified GDP growth:")
print(df_america_oceania[['Country', 'GDP growth', 'Economic_growth_level']])
```

```
7                    Brazil   -3.096007          Very Low
8                    Canada    2.238638               Low
9                     Chile   -4.372474          Very Low
10                 Colombia   -6.760440          Very Low
11               Costa Rica   -5.831053          Very Low
12                     Cuba   -2.065512          Very Low
13                 Dominica   -2.065512          Very Low
14        Dominican Republic  -12.036793          Very Low
15                  Ecuador   -1.857289          Very Low
16               El Salvador   -4.278604          Very Low
17                      Fiji   -1.606636          Very Low
18                  Grenada   -2.500000          Very Low
19                Guatemala   -8.855279          Very Low
20                   Guyana   -8.398364          Very Low
21                    Haiti    0.503032          Very Low
22                 Honduras  -21.399900          Very Low
23                  Jamaica   -5.534435          Very Low
24                 Kiribati  -33.492796          Very Low
25          Marshall Islands   -4.557752          Very Low
26                   Mexico    3.173774            Medium
```

```
Missing values in American countries data:
Official language                                3
Currency-Code                                    3
Agricultural Land( )                             1
Birth Rate                                       2
Minimum wage                                     8
Population: Labor force participation ()        10
Tax revenue ()                                   8
Total tax rate                                   3
Unemployment rate                               10
Urban_population                                 1
dtype: int64
```

Stream 2

STEP 1:

IN THIS STEP, WE LOAD THE DATASET INTO PYTHON AND TAKE A FIRST LOOK AT ITS STRUCTURE. WE START BY IMPORTING THE NECESSARY LIBRARIES: PANDAS FOR HANDLING TABLES AND NUMPY FOR CALCULATIONS. THEN, WE USE PD.READ_EXCEL("BASEDEDONNEE2.XLSX") TO LOAD THE DATA FROM AN EXCEL FILE INTO A TABLE-LIKE FORMAT CALLED A DATAFRAME, STORED IN THE VARIABLE DATA. FINALLY, WE USE DATA.HEAD() TO SEE THE FIRST FEW ROWS OF THE DATASET, HELPING US UNDERSTAND ITS LAYOUT, COLUMN NAMES, AND CHECK FOR ANY OBVIOUS PROBLEMS LIKE MISSING OR INCORRECT DATA.

THE CODE:

```python
import pandas as pd
import math
import matplotlib.pyplot as plt

# Step 1: Load the Dataset
file_path = "basededonnee2.xlsx"  # Replace with your file path
df = pd.read_excel(file_path)

# Replace 'Population' with the column name containing the continuous variable
column_name = "GDP growth"
df[column_name] = pd.to_numeric(df[column_name], errors='coerce')
data = df[column_name].dropna().tolist()  # Clean and convert to list

if not data:
    print("No valid data found in the selected column.")
else:
    print("Data Loaded Successfully:", data)
```

STEP 2:

TO DEFINE CLASSES AND FREQUENCIES, APPLY STURGES' RULE: K = 1 + 3.322 LOG(N), WHERE N IS THE NUMBER OF OBSERVATIONS. DIVIDE THE DATA RANGE INTO EQUAL INTERVALS BASED ON K. THEN, COUNT THE DATA POINTS IN EACH INTERVAL TO DETERMINE THE FREQUENCIES. FINALLY, CALCULATE RELATIVE FREQUENCIES BY DIVIDING EACH FREQUENCY BY THE TOTAL SIZE AND CUMULATIVE FREQUENCIES BY SUMMING THEM PROGRESSIVELY.

THE CODE:

```
#  Build a function to define the classes manually
class_dict = {}
for continent in unique_classes:
    class_dict[continent] = []  # Initialize empty list for each class

for index, row in df.iterrows():
    continent = row[qualitative_variable]
    # Manually add the row to the correct continent class
    for continent_key in class_dict:
        if continent == continent_key:
            class_dict[continent_key].append(row)
            break
```

STEP 3:

CALCULATE KEY STATISTICS, SUCH AS CLASS CENTERS, MEAN (USING MIDPOINTS), VARIANCE, AND STANDARD DEVIATION. THE COEFFICIENT OF VARIATION MEASURES DISPERSION, WHILE SKEWNESS SHOWS ASYMMETRY AND KURTOSIS REFLECTS TAIL BEHAVIOR. THESE METRICS OFFER A THOROUGH ANALYSIS OF DATA DISTRIBUTION AND SPREAD.

THE CODE:

```python
# Perform Calculations and Display Results
if data:
    num_classes = calculate_classes(data)
    classes = define_classes(data, num_classes)
    frequencies, relative_frequencies, cumulative_frequencies = calculate_frequencies(data, classes)
    class_centers = calculate_class_centers(classes)
    mean = calculate_weighted_mean(class_centers, frequencies)
    variance = calculate_variance(class_centers, frequencies, mean)
    std_dev = math.sqrt(variance)
    skewness = calculate_skewness(class_centers, frequencies, mean, std_dev)
    kurtosis = calculate_kurtosis(class_centers, frequencies, mean, std_dev)
    coefficient_of_variation = (std_dev / mean * 100) if mean != 0 else 0

    # Create Statistical Table for Classes
    table_data = {
        'Class Interval': [f"[{round(lower, 2)}, {round(upper, 2)})" for lower, upper in classes],
        'Frequency': frequencies,
        'Relative Frequency': [round(rf, 4) for rf in relative_frequencies],
        'Cumulative Frequency': cumulative_frequencies}
    stats_df = pd.DataFrame(table_data)
```

STEP 4:

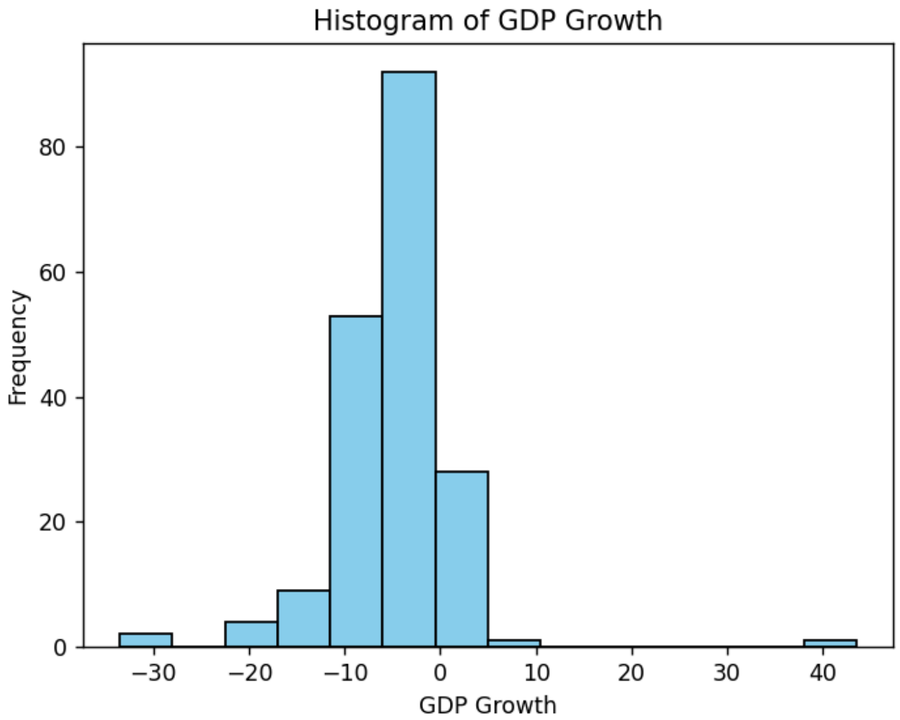
GENERATE BAR AND PIE CHARTS TO EFFECTIVELY VISUALIZE DATA DISTRIBUTION. ANALYZE AND INTERPRET KEY INSIGHTS, INCLUDING THE MODE, DATA SYMMETRY, AND THE EXTENT OF VARIABILITY.

THE CODE:

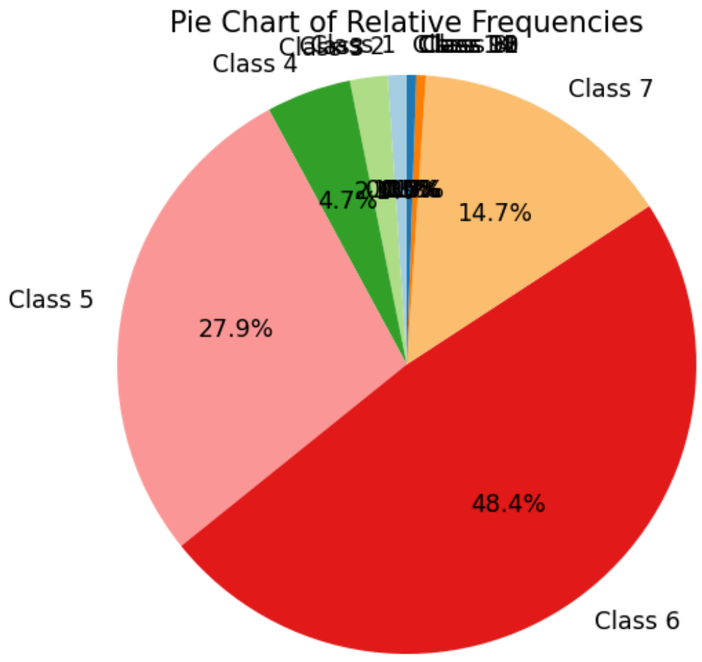
```python
# Plot Histogram
def plot_histogram(data, num_classes=10, variable_name="GDP Growth"):
    plt.hist(data, bins=num_classes, color='skyblue', edgecolor='black')
    plt.title(f"Histogram of {variable_name}")
    plt.xlabel(variable_name)
    plt.ylabel("Frequency")
    plt.show()

# Plot Pie Chart
def plot_pie_chart(relative_frequencies):
    labels = [f"Class {i+1}" for i in range(len(relative_frequencies))]
    plt.pie(relative_frequencies, labels=labels, autopct='%1.1f%%', startangle=90, colors=plt.cm.Paired.colors)
    plt.title("Pie Chart of Relative Frequencies")
    plt.axis('equal')  # Equal aspect ratio ensures the pie chart is circular.
    plt.show()

# Display Plots
plot_histogram(data, num_classes, variable_name="GDP Growth")
plot_pie_chart(relative_frequencies)
```



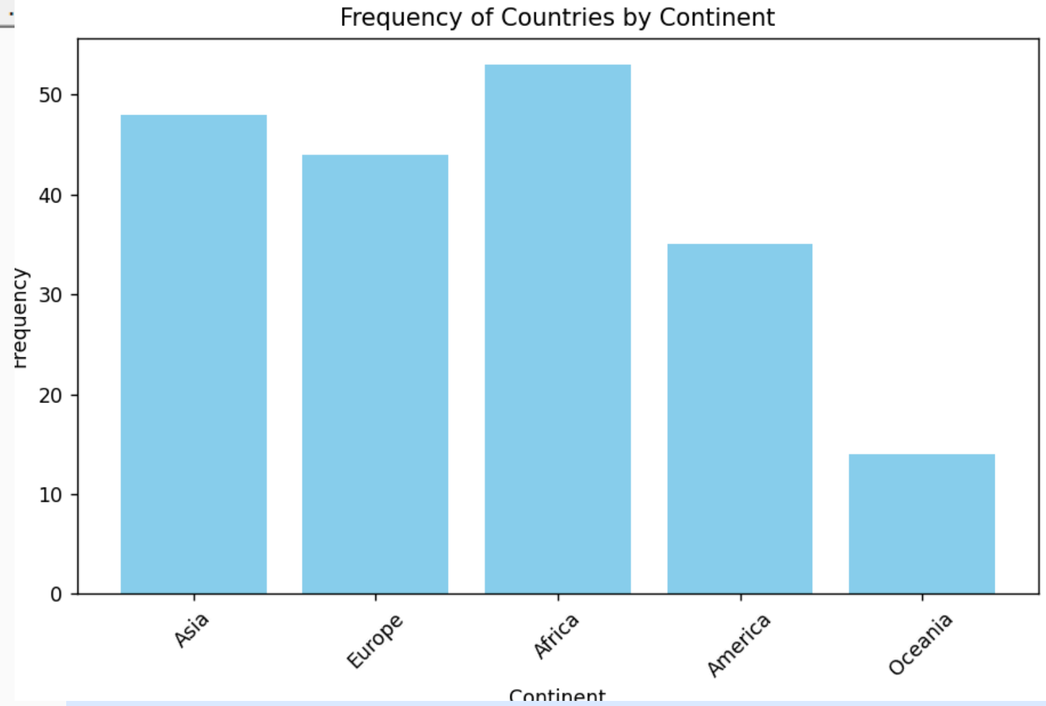
THE RESULT:




```
Data Loaded Successfully: [-2.35110067344623, -3.30208203862541, -5.10000000018678, -11.18393993 .

Class Frequency Table:
      Class Interval  Frequency  Relative Frequency  Cumulative Frequency
0   [-33.49, -27.99)          2              0.0105                     2
1    [-27.99, -22.5)          0              0.0000                     2
2     [-22.5, -17.0)          4              0.0211                     6
3     [-17.0, -11.5)          9              0.0474                    15
4      [-11.5, -6.0)         53              0.2789                    68
5       [-6.0, -0.5)         92              0.4842                   160
6       [-0.5, 4.99)         28              0.1474                   188
7      [4.99, 10.49)          1              0.0053                   189
8     [10.49, 15.99)          0              0.0000                   189
9     [15.99, 21.49)          0              0.0000                   189
10    [21.49, 26.99)          0              0.0000                   189
11    [26.99, 32.48)          0              0.0000                   189
12    [32.48, 37.98)          0              0.0000                   189
13    [37.98, 43.48)          1              0.0053                   190

Additional Statistical Measures:
Skewness: 0.9850 - Positively Skewed
Kurtosis: 14.8669 - Leptokurtic
Coefficient of Variation: -131.48%
```



CONCLUSION

IN THIS PROJECT, WE SYSTEMATICALLY PROCESSED THE DATASET BY LOADING, INSPECTING, AND CLEANING IT TO ADDRESS MISSING VALUES AND ENSURE ACCURACY. WE EXPLORED PATTERNS AND TRENDS THROUGH STATISTICAL ANALYSIS AND VISUALIZATIONS, AND ANALYZED THE RESULTS TO ANSWER KEY QUESTIONS, SHOWCASING HOW RAW DATA CAN BE CONVERTED INTO ACTIONABLE INSIGHTS.

Stream 3
Bivariate Analysis of Tax Revenue

STEP 1: Calculating Covariance and Correlation

In this step we need to<u>Load the Excel Dataset</u>;Using the pandas library to read data from an Excel file.
<u>Convert Data to Numeric</u>;you need to ensure the data in the "Tax revenue" column is numeric using pd.to_numeric.;Handle invalid or missing values using the errors='coerce' option.
<u>Clean the Data</u> in order to do that you need to Loop through the column and keep only valid (non-NaN) entries.

THE CODE:

```
import pandas as pd

file_path = "basededonnee2.xlsx"  # File path
column_name = "Tax revenue ()"    # Target column

# Load dataset
df = pd.read_excel(file_path)

# Convert column to numeric and remove invalid entries
df[column_name] = pd.to_numeric(df[column_name], errors='coerce')
cleaned_data = df[column_name].dropna().tolist()

print(f"Data Loaded Successfully with {len(cleaned_data)} valid entries.")
```

STEP 2: Calculate Covariance and Correlation

THE CODE:

```
import numpy as np

# Create index data (X) and clean values (Y)
x = np.arange(len(cleaned_data))
y = cleaned_data

# Compute covariance
mean_x = np.mean(x)
mean_y = np.mean(y)
covariance = np.sum((x - mean_x) * (y - mean_y)) / len(x)

# Compute correlation
std_x = np.std(x)
std_y = np.std(y)
correlation = covariance / (std_x * std_y)

print(f"Covariance: {covariance}")
print(f"Correlation: {correlation}")
```

<u>Prepare Index Data:</u>

Use np.arange() to create an index corresponding to data points in the list (Tax Revenue).

<u>Calculate Covariance:</u>

Use loops to compute covariance

<u>Calculate Correlation:</u>

Compute standard deviations,then divide covariance by their product.

STEP 3: Calculate Linear Regression Coefficients

Calculate Slope :

Divide covariance by the variance

Calculate Intercept:

Use the formula a=y¯−b·x¯a = \bar{y} - b \cdot \bar{x}a=y¯−b·x¯.

Store Results for Plotting:

Use the slope and intercept to create the regression line.

THE CODE:

```
# Calculate slope (b) and intercept (a)
b = covariance / np.var(x)
a = mean_y - b * mean_x


print(f"Linear Regression Coefficients: a = {a}, b = {b}")
```

STEP 4: Visualize the Data and Regression Line

THE CODE:

```
import matplotlib.pyplot as plt

# Scatter plot for data points
plt.scatter(x, y, color='blue', label='Data Points')

# Regression line
regression_line = [a + b * xi for xi in x]
plt.plot(x, regression_line, color='red', label='Regression Line')

# Add labels and legend
plt.xlabel('Index (Time or Observation)')
plt.ylabel('Tax Revenue (%)')
plt.title('Scatter Plot with Linear Regression Line')
plt.legend()
plt.show()
```

Create a Scatter Plot:

Use plt.scatter() to plot the original data points.

Plot the Regression Line:

Use the linear regression equation y=a+b·xy = a + b \cdot xy=a+b·x to compute and plot the line.

Add Labels and Legends:

Enhance the plot with axis labels, a title, and a legend.

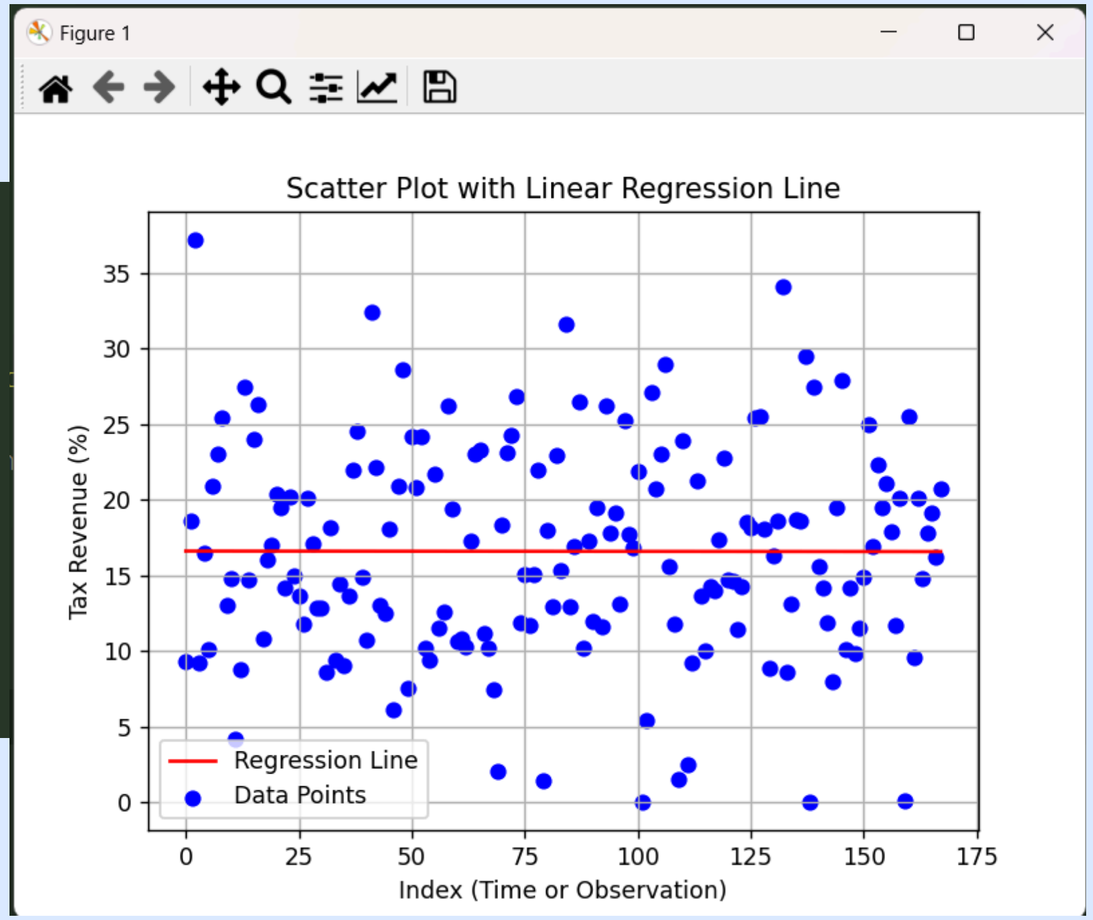
Output:

```
>>> %Run Pl.py

Data Loaded Successfully with 168 valid entries.

--- Results ---
Mean X: 83.50, Mean Y: 16.59
Variance X: 2351.92, Variance Y: 48.44
Covariance: -0.52
Correlation: -0.00
Linear Regression Coefficients: a (intercept) = 16.60, b (slope) = -0.00

>>>
```



Conclusion:

Our analysis showed the relationship between index (X) and tax revenue (Y) using Python. By calculating key metrics like covariance, correlation, and regression coefficients, we were able to interpret the trend and strength of this relationship visually through scatter plots and regression lines. This analysis demonstrates how data techniques can reveal insights and guide decision-making

Conclusion:

The analysis demonstrated how Python can be used to explore the relationship between index (X) and tax revenue (Y). By calculating key metrics such as covariance, correlation, and regression coefficients, we were able to interpret the trend and strength of this relationship effectively. The visual representation through scatter plots and regression lines allowed us to understand and predict trends in the data.

Thanks for your attention!